

Comparative Evaluation of Trajectory Optimization Methods for Ballistic Throwing with a Planar Robotic Arm in Obstacle-Constrained Environments

Jash Shah

ECE 594T: Optimization and Learning for Robotics and Control

Dept. of Electrical and Computer Engineering, UCSB

jash.shah@ucsb.edu

Abstract—This paper presents a comparative study of six trajectory optimization approaches for an object throwing task with a 4-link planar robot arm toward a target on the ground ($d = 2.977$ m) while navigating through obstacles. The approaches include three transcription methods—direct shooting (M1–M2), direct collocation (M3–M4), and sequential convex programming (M5–M6)—in standard and GCS warm-started variants. All approaches were tested against three obstacle configurations (Simple, Moderate, Hard). Results indicate: (1) SCP (M5) achieves ≤ 1.3 cm landing error on all configurations, a $40\text{--}600\times$ improvement over shooting/collocation; (2) GCS warm-starting improves speeds by $1.5\text{--}2.4\times$ for collocation (M4) but degrades convergence for direct shooting (M2) and SCP (M6) due to initialization–transcription mismatch; (3) direct collocation (M4) offers the best balance between speed and accuracy when SCP convergence is questionable. Full system modeling, mathematical formulations for each method, and a detailed analysis of failure modes are provided. Source code: https://github.com/Jash-2000/Robotics_Optimization.

Index Terms—trajectory optimization, robotic throwing, direct collocation, sequential convex programming, graph of convex sets, obstacle avoidance, Lagrangian dynamics

I. INTRODUCTION

Robotic throwing requires high-speed manipulation specified by an exact terminal condition: the object velocity upon release must produce a ballistic trajectory reaching a distant target. Unlike positioning tasks, it requires simultaneous satisfaction of torque limits, collision avoidance, and a precise projectile objective, making it an ideal benchmark for comparing optimization paradigms. Several papers examine certain approaches individually [1]– [5]; however, cross-paradigm comparisons on the same nonlinear obstacle-constrained system are uncommon. This work fills that gap.

Contributions. (i) A complete Lagrangian dynamics model for a 4-link planar arm with attached object, derived symbolically and verified by energy conservation. (ii) Explicit mathematical formulations for all six optimization methods, sharing a common canonical OCP. (iii) A quantified comparison of GCS warm-start benefits across three transcription paradigms. (iv) Root-cause analysis of SCP collision persistence with five identified mechanisms. (v) Detailed implementation notes and design decisions for reproducibility.

TABLE I: Link parameters for the 4-link planar arm.

Link	Role	Length (m)	Mass (kg)	Inertia
1	Shoulder	0.40	2.0	$\frac{1}{12}m_1l_1^2$
2	Forearm	0.35	1.5	$\frac{1}{12}m_2l_2^2$
3	Wrist	0.30	1.0	$\frac{1}{12}m_3l_3^2$
4	Gripper	0.08	0.3	$\frac{1}{12}m_4l_4^2$

II. SYSTEM MODELING

A. Robot Description

A 4-link planar arm (Table I) is mounted at height $y_0 = 1.0$ m above the ground. The arm grasps a rectangular object (0.12×0.06 m, $m_{\text{obj}} = 0.5$ kg, $I_{\text{obj}} = \frac{1}{3}m_{\text{obj}}(a^2 + b^2)$) and must throw it to a target at $d = 2.977$ m.

B. Forward Kinematics

Joint angles are relative: $\mathbf{q} = [q_1, q_2, q_3, q_4]^T$. Absolute angles are $\alpha_i = \sum_{k=1}^i q_k$. The position of joint i in the world frame is:

$$\mathbf{p}_{\text{joint},i} = \mathbf{p}_{\text{joint},i-1} + l_{i-1} \begin{bmatrix} \cos \alpha_{i-1} \\ \sin \alpha_{i-1} \end{bmatrix}, \quad \mathbf{p}_{\text{joint},0} = \begin{bmatrix} 0 \\ y_0 \end{bmatrix} \quad (1)$$

The link center-of-mass is $\mathbf{p}_{\text{CoM},i} = \mathbf{p}_{\text{joint},i} + \frac{l_i}{2} [\cos \alpha_i, \sin \alpha_i]^T$. The object CoM includes a fixed gripper-to-CoM offset $\mathbf{r}_{\text{gc}} = [0.05, 0]^T$ m in the gripper body frame:

$$\mathbf{p}_{\text{obj}} = \mathbf{p}_{\text{ee}} + \mathbf{R}(\alpha_4) \mathbf{r}_{\text{gc}} \quad (2)$$

C. Lagrangian Dynamics

The kinetic energy includes all link and object contributions:

$$T = \sum_{i=1}^4 \left[\frac{1}{2} m_i \|\dot{\mathbf{p}}_{\text{CoM},i}\|^2 + \frac{1}{2} I_i \dot{\alpha}_i^2 \right] + \frac{1}{2} m_{\text{obj}} \|\dot{\mathbf{p}}_{\text{obj}}\|^2 + \frac{1}{2} I_{\text{obj}} \dot{\alpha}_4^2 \quad (3)$$

The potential energy is $V = \sum_i m_i g p_{\text{CoM},i,y} + m_{\text{obj}} g p_{\text{obj},y}$. The Euler–Lagrange equations yield the manipulator form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau} \quad (4)$$

where $\mathbf{M} \in \mathbb{R}^{4 \times 4}$ is symmetric positive-definite, $\mathbf{C}$ is constructed via Christoffel symbols $C_{ij} = \sum_k \Gamma_{ijk} \dot{q}_k$ with $\Gamma_{ijk} = \frac{1}{2}(\partial M_{ij}/\partial q_k + \partial M_{ik}/\partial q_j - \partial M_{jk}/\partial q_i)$, and $G_i = \partial V/\partial q_i$. This construction guarantees the skew-symmetry property $\dot{\mathbf{M}} - 2\mathbf{C}$ is skew-symmetric, which is verified numerically.

Validation. Energy conservation is checked with $\tau = 0$ for 2 s: $|E(t) - E(0)|/|E(0)| < 0.001$ (PASS).

D. Geometric Jacobians

The object velocity at release is computed via the geometric Jacobian:

$$\mathbf{v}_{\text{obj}} = \mathbf{J}_{\text{obj}}(\mathbf{q})\dot{\mathbf{q}}, \quad \omega_{\text{obj}} = \sum_{i=1}^4 \dot{q}_i \quad (5)$$

where column i of $\mathbf{J}_{\text{obj}}$ is the 2D cross product of the z -axis with the vector from joint i to the object CoM.

E. Hybrid Dynamics: Release and Ballistic Phase

The simulation has two phases separated by the release event at $t = t^*$.

Phase 1 (Manipulation, $t \in [0, t^*]$): Arm carries the object; dynamics governed by (4) with object inertia included.

Phase 2 (Ballistic, $t \in [t^*, t_{\text{land}}]$): Object released; follows projectile motion:

$$\mathbf{r}(t) = \mathbf{r}_0 + \mathbf{v}_0 t + \frac{1}{2}[0, -g]^T t^2 \quad (6)$$

$$\theta(t) = \theta_0 + \omega_0 t \quad (7)$$

Landing time: $t_{\text{land}} = (v_{0,y} + \sqrt{v_{0,y}^2 + 2g r_{0,y}})/g$, and landing position: $x_{\text{land}} = r_{0,x} + v_{0,x} t_{\text{land}}$.

F. Obstacle Configurations

Three fixed configurations provide reproducible benchmarks (Fig. 2):

G. Collision Detection

Two classes of collision are checked: (i) *arm–obstacle*: each link modeled as a line segment, checked against obstacle circles via minimum point-to-segment distance; (ii) *object–obstacle*: the oriented bounding rectangle at position $\mathbf{r}(t)$ and orientation $\theta(t)$ is checked using the Separating Axis Theorem (SAT) with 4 candidate axes. Signed distances are returned for both hard-constraint and penalty formulations.

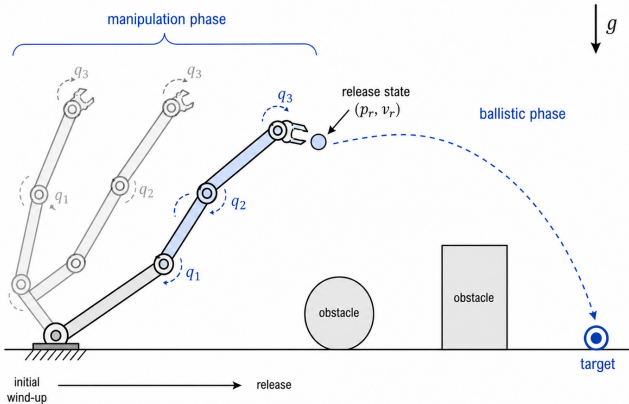


Fig. 1: The overall idea of what this optimization problem is solving. I consider a 2-D environment with a planar robotic arm throwing objects to a target location.

III. PROBLEM FORMULATION

A. Canonical Optimal Control Problem

Let T be the number of nodes and $h = t^*/T$. The canonical OCP is:

$$\min_{\mathbf{x}_{1:T}, \boldsymbol{\tau}_{1:T}, t^*} J(\mathbf{x}_{1:T}, \boldsymbol{\tau}_{1:T}) \quad (8)$$

$$\text{s.t. } \mathbf{x}_1 = [\frac{\pi}{2}, 0, 0, 0, 0, 0, 0]^T \quad (9)$$

$$\mathbf{x}_{k+1} = \Phi(\mathbf{x}_k, \boldsymbol{\tau}_k, h), \quad k = 1, \dots, T-1 \quad (10)$$

$$\boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}_k \leq \boldsymbol{\tau}_{\max} \quad (11)$$

$$|\dot{q}_{i,k}| \leq 10 \text{ rad/s}, \quad |q_{i,k}| \leq \pi \quad (12)$$

$$c_{\text{arm}}(\mathbf{x}_k) \geq 0, \quad c_{\text{flight}}(\mathbf{x}_T) \geq 0 \quad (13)$$

$$0.8 \text{ s} \leq t^* \leq 2.0 \text{ s} \quad (14)$$

with asymmetric torque limits $\boldsymbol{\tau}_{\max} = [20, 15, 10, 2]^T \text{ N m}$.

B. Objective Function

$$J = \underbrace{w_p(x_{\text{land}}(\mathbf{x}_T) - d)^2}_{\text{landing accuracy}} + \underbrace{w_E \sum_{k=1}^T \|\boldsymbol{\tau}_k\|^2 h}_{\text{energy}} + \underbrace{w_c J_{\text{collision}}}_{\text{collision penalty}} \quad (15)$$

Landing accuracy dominates ($w_p = 200$ – 1000); energy prevents excessive torques ($w_E = 0.1$ – 1). An additional direction penalty $w_{\text{dir}} \max(0, -v_{x,\text{release}})$ prevents backward throws.

C. Collision Constraints

For each circular obstacle j with center $\mathbf{c}_j$ and radius r_j , the signed distance at check-point $\mathbf{p}_i(\mathbf{q}_k)$ is:

$$c_{ijk} = \|\mathbf{p}_i(\mathbf{q}_k) - \mathbf{c}_j\| - r_j - \delta \geq 0 \quad (16)$$

where $\delta = 0.05$ – 0.20 m is a safety margin. For SCP, this is linearized as a halfplane plus slack:

$$\hat{n}_{kj}^T [\mathbf{p}_i(\bar{\mathbf{q}}_k) + J_{\mathbf{p}_i}(\bar{\mathbf{q}}_k)(\mathbf{q}_k - \bar{\mathbf{q}}_k) - \mathbf{c}_j] \geq r_j + \delta - s_{kj} \quad (17)$$

IV. METHOD I: DIRECT SHOOTING (M1/M2)

A. Formulation

Decision variables $\mathbf{z} \in \mathbb{R}^{41}$: $T = 10$ piecewise-constant torque intervals plus t^* . The dynamics constraint (10) is enforced *implicitly* by integrating (4) with ode45; no state variables appear in $\mathbf{z}$. Torques are constant within each interval:

$$\boldsymbol{\tau}(t) = \boldsymbol{\tau}^{(k)} \quad \text{for } t \in [(k-1)\Delta t, k\Delta t], \quad \Delta t = t^*/T \quad (18)$$

The objective is a weighted sum: $J = 1000(x_{\text{land}} - d)^2 + E_{\text{total}}/t^* + 10 \max(0, -v_x)$. Collision constraints use the hard form of (16) passed to fmincon as nonlinear inequalities at 50 sampled arm poses and 100+ ballistic-arc points.

B. Solver Configuration

Interior-point algorithm with forward finite differences (step size 10^{-6}), maximum 200 iterations, tolerance 10^{-4} . Each gradient evaluation requires 41 ODE solves, making each iteration expensive ($\approx 90 \text{ s}$ for gradient computation).

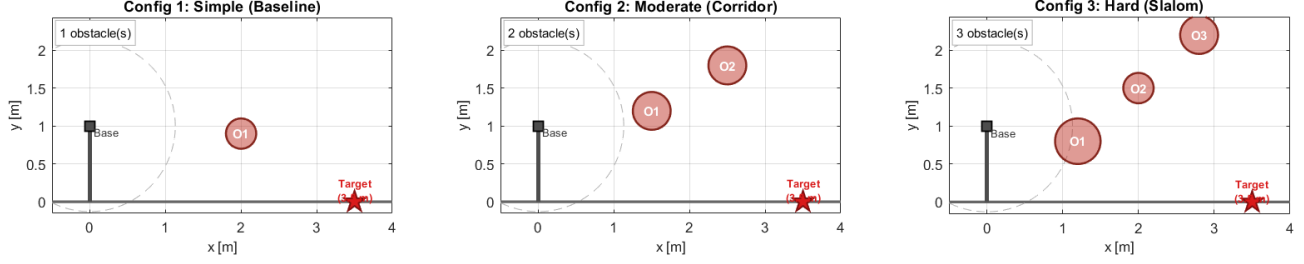


Fig. 2: Three fixed obstacle configurations. *Simple*: 1 circle at (1.3, 0.9) m, $r = 0.20$ m. *Moderate*: 2 circles forming a corridor at (1.0, 1.2) and (1.7, 1.8) m, $r = 0.25$ m. *Hard*: 3-obstacle slalom at (0.8, 0.8) m ($r = 0.30$), (1.4, 1.5) m ($r = 0.20$), (1.9, 2.2) m ($r = 0.25$). Dashed arc = reachable workspace; star = target.

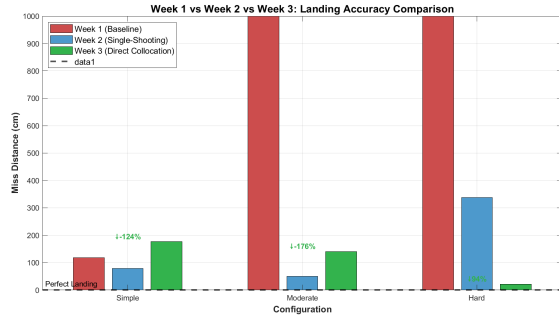


Fig. 3: Landing accuracy: Week 1 vs. Week 2 vs. Week 3. Week 2 (shooting) improves simple/moderate; Week 3 (collocation) dramatically improves hard.

C. Initialization

M1 uses a physics-informed heuristic: Week 1 sinusoidal baseline $\tau_i(t) = A_i \sin(\omega_i t + \phi_i)$ sampled at interval centers with 1% noise. M2 substitutes a GCS-derived guess (Section VII).

D. Key Design Decisions

Weight scale matters for finite differences. $w_p = 100$ produced unreliable gradients because the objective magnitude was too small relative to the finite-difference step; increasing to $w_p = 1000$ resolved this.

Direction penalty. Without it, the optimizer found backward-throwing solutions (release angle ≈ 135). Adding $w_{\text{dir}} = 10$ eliminated these pathological cases.

Release time bounds. Tightened from $[0.5, 2.0]$ s to $[0.8, 1.5]$ s to prevent unrealistic trajectories.

V. METHOD II: DIRECT COLLOCATION (M3/M4)

A. Formulation

Decision variables $\mathbf{z} \in \mathbb{R}^{721}$: joint angles $\mathbf{q}_k$, velocities $\dot{\mathbf{q}}_k$, and torques $\boldsymbol{\tau}_k$ at $T = 60$ nodes, plus t^* . Dynamics are enforced via $2N(T-1) = 472$ trapezoidal defect equalities:

$$\mathbf{q}_{k+1} - \mathbf{q}_k - \frac{h}{2}(\dot{\mathbf{q}}_k + \dot{\mathbf{q}}_{k+1}) = 0 \quad (19)$$

$$\dot{\mathbf{q}}_{k+1} - \dot{\mathbf{q}}_k - \frac{h}{2}(\ddot{\mathbf{q}}_k + \ddot{\mathbf{q}}_{k+1}) = 0 \quad (20)$$

where $\ddot{\mathbf{q}}_k = \mathbf{M}^{-1}(\boldsymbol{\tau}_k - \mathbf{C}\dot{\mathbf{q}}_k - \mathbf{G})$.

B. Collision Handling

Collision avoidance is implemented as a *penalty method* with weight $w_c = 10^8$:

$$J_{\text{col}} = \sum_{k=1}^T \sum_j \sum_s \max(0, (r_j + \delta)^2 - \|\mathbf{p}_s - \mathbf{c}_j\|^2)^2 \quad (21)$$

evaluated at three sample points per link (endpoints + mid-point) and at the object CoM. The flight arc is sampled at 40 points with a quartic penalty for stronger gradient response. This penalty approach was chosen because CasADi's Opti interface requires symbolic constraint expressions, which complicates standard collision-checking functions.

C. Solver

CasADi provides exact sparse Jacobians and Hessians via automatic differentiation. IPOPT [4] solves the resulting NLP using an interior-point filter line-search method. The sparse structure (721 variables, 480 equality constraints) is exploited for efficient factorization.

D. Initialization

M3 uses a collision-aware initial guess: the trajectory is simulated step-by-step, applying corrective torques when imminent collisions are detected, gradually building forward velocity while maintaining a collision-free path. M4 substitutes a GCS-derived guess.

VI. METHOD III: SEQUENTIAL CONVEX PROGRAMMING (M5/M6)

A. Formulation

Decision variables $\mathbf{z} \in \mathbb{R}^{720}$: states and torques at $T = 60$ nodes; t^* is handled separately. At each SCP iteration i , given reference $(\bar{\mathbf{x}}_k, \bar{\boldsymbol{\tau}}_k)$, dynamics are linearized:

$$\mathbf{x}_{k+1} = (\mathbf{I} + h\mathbf{A}_k)\mathbf{x}_k + h\mathbf{B}_k\boldsymbol{\tau}_k + h(\mathbf{f}_k - \mathbf{A}_k\bar{\mathbf{x}}_k - \mathbf{B}_k\bar{\boldsymbol{\tau}}_k) \quad (22)$$

where $\mathbf{A}_k = \partial f / \partial \mathbf{x}|_{\bar{\mathbf{x}}_k}$ and $\mathbf{B}_k = \partial f / \partial \boldsymbol{\tau}|_{\bar{\boldsymbol{\tau}}_k}$ are computed via central finite differences (24 evaluations per step, $\mathcal{O}(\epsilon^2)$ accuracy). Since f is linear in $\boldsymbol{\tau}$, $\mathbf{B}_k = [0_{4 \times 4}; \mathbf{M}(\mathbf{q})^{-1}]$ has sparse structure.

B. QP Sub-Problem

$$J_{QP} = w_p(x_{\text{land,lin}} - d)^2 + w_E \sum_k \|\tau_k\|^2 h + w_\rho \sum_k \|\mathbf{x}_k - \bar{\mathbf{x}}_k\|^2 + w_s \sum_j \tau s_j^2 \quad (23)$$

Trust-region constraints: $\|\mathbf{x}_k - \bar{\mathbf{x}}_k\|_\infty \leq \rho$, $\|\tau_k - \bar{\tau}_k\|_\infty \leq \rho_u$. Collision avoidance via linearized halfplanes (17) with slack variables $s_{kj} \geq 0$.

C. Release Time Update

t^* is excluded from the QP. Including it creates a zero diagonal in the Hessian (no quadratic cost on t^*), rendering H indefinite and causing `quadprog` to fail with `exitflag -2`. Instead, t^* is updated via golden-section search:

$$t^{*(i+1)} = \arg \min_{t^* \in [0.8, 2.5]} J(\mathbf{x}^{(i+1)}, \tau^{(i+1)}, t^*) \quad (24)$$

D. Trust Region and Step Acceptance

After each QP solve, a backtracking line search with $\alpha \in \{1.0, 0.5, 0.25, 0.125\}$ is performed. A step is accepted only if $J(\mathbf{x}_{\text{new}}) < J(\bar{\mathbf{x}})$. This hard rule prevents the pathological case where both numerator and denominator of the trust ratio ξ are negative, which produces a spuriously positive ratio while the cost actually increases.

Trust ratio $\xi = (J_{\text{old}} - J_{\text{new}})/(J_{\text{old}} - J_{\text{QP}}^*)$ governs radius updates: $\xi > 0.75 \Rightarrow \text{expand } \rho$; $\xi \leq 0.25 \Rightarrow \text{contract } \rho$.

E. Key Design Decisions

- 1) **Forward Euler** (not trapezoidal) for SCP dynamics: linearization around the reference makes the defect constraint affine, which is not guaranteed with implicit schemes.
- 2) **No joint-angle hard bounds**: the backward-swing trajectory spans $q \in [-25.9, 8.5]$ rad, far outside $[-\pi, \pi]$; enforcing bounds creates $\text{lb} > \text{ub}$, making the QP infeasible.
- 3) **RK4 rollouts**: Forward Euler diverges at $h \approx 0.033$ s with the arm's stiff dynamics.
- 4) **Best-solution tracking**: the runner stores the best solution seen across all iterations, returning it even if later iterations degrade.

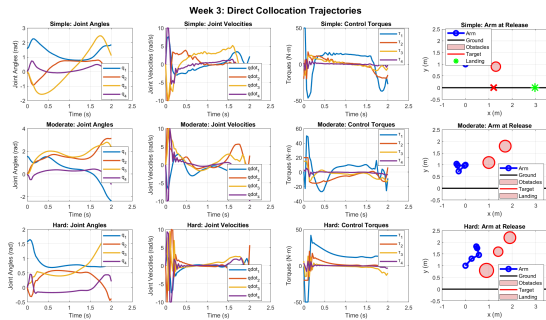


Fig. 4: Week 3 direct collocation trajectories. Each row = one configuration (simple, moderate, hard). Columns: joint angles, velocities, torques, and arm pose at release. The hard configuration shows smooth torque profiles that navigate all three obstacles.

TABLE II: Final SCP parameters for M5 and M6.

Parameter	M5	M6	Notes
Max iters	60	60	Cross-method fairness
ρ (state)	50	50	Converges in 14–19
ρ_u (control)	0.5	1.0	M6 larger: GCS near-feasible
w_p	5.0	8.0	
w_E	200	200	
w_ρ (trust)	0.1	0.1	
w_s (slack)	5.0	5.0	
w_{coll}	10^4	10^4	
δ (margin)	10^7	10^8	M6 needs stronger penalty
	0.20 m	0.12 m	

5) **Slack variables**: added to every collision inequality to ensure QP feasibility when the reference intersects obstacles.

F. SCP Parameters

VII. GCS WARM-STARTING PIPELINE

GCS [6] decomposes the 2-D workspace into convex obstacle-free regions and solves a Dijkstra shortest-path problem through the region graph. The 6-step repair bridge converts the geometric path into a dynamically-feasible initial guess:

- 1) **Region decomposition**: axis-aligned rectangular cells tile the obstacle-free workspace; obstacles dilated by 5 cm. All regions clipped to the arm's reachable annulus ($L_{\text{total}} = 1.13$ m). Degenerate regions ($x_{\min} \geq x_{\max}$) trigger an error.
- 2) **Graph search**: Dijkstra on cell-adjacency graph with Euclidean edge weights; $K = 5$ perturbed PCHIP-smoothed paths.
- 3) **Inverse kinematics**: damped Jacobian pseudoinverse ($\Delta \mathbf{q} = J^T(JJ^T + \lambda^2 I)^{-1} \mathbf{e}$, 200 iterations, $\lambda = 0.01$) with `fmincon` fallback. Unreachable targets clamped to $0.95L_{\text{total}}$.
- 4) **Collision validation**: each candidate checked; best collision-free one selected.
- 5) **Time-scaling**: torques computed via inverse dynamics; if $\max |\tau_i| > \tau_i^{\max}$, time-scaled by $\alpha = \sqrt{\tau_{\text{actual}}/\tau_{\text{limit}}}$. Throwing swing appended.
- 6) **Format packaging**: $(q_k, \dot{q}_k, \tau_k)$ for M4; piecewise-constant torques for M2; RK4-rollout states for M6.

Key property: The GCS path is geometrically valid but dynamically relaxed—it respects obstacle clearance by construction but does not encode Lagrangian dynamics.

Timing: GCS initialization completes in < 1 s (0.19–0.61 s), representing 2–7% of M4's total solve time.

VIII. RESULTS

A. Baseline (Week 1)

With fixed sinusoidal torques $\tau_i(t) = A_i \sin(\omega_i t + \phi_i)$, the arm achieves 117.8 cm miss on simple but >1000 cm (infeasible) on moderate/hard. This establishes the need for optimization.

B. Week 1 Simulation Snapshots

Figs. 5–6 show complete manipulation and ballistic trajectories for each configuration with fixed torques.

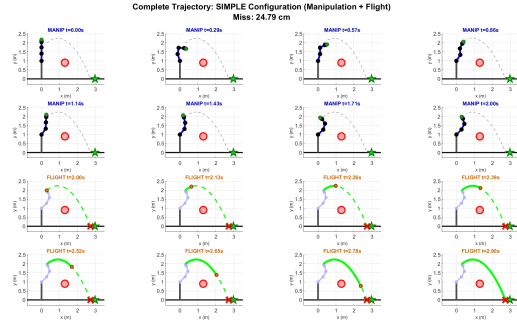


Fig. 5: Week 1 baseline: Simple configuration. Miss = 24.79 cm. The manipulation phase (top two rows) shows the arm swinging, and the ballistic flight (bottom two rows) shows the object arc toward the target.

C. M1/M2 Results (Direct Shooting)

M1 achieves 78.9 cm (simple), 50.9 cm (moderate), 337.8 cm (hard). Hard config reaches `fmincon` exitflag -2 , indicating the 10-interval piecewise control cannot represent the complex arm maneuvers needed.

D. M3/M4 Results (Direct Collocation)

M3 achieves 177 cm (simple), 140 cm (moderate), 21 cm (hard). Notably, M3 is *best* on hard—the 60-node state/control discretization captures complex maneuvers that shooting cannot.

E. Full 6-Method Comparison

Table III presents the complete 6×3 results matrix.

F. Solve Time and Iteration Analysis

Figures 8, 9 and 10 talk about the differences.

G. Convergence Behavior

Figure 11 illustrates the overall convergent behaviour of the design.

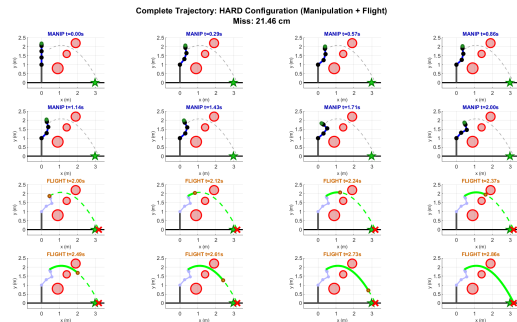


Fig. 6: Week 1 baseline: Hard configuration. Miss = 21.46 cm. The flight arc passes close to obstacle O3 at (1.9, 2.2) m.

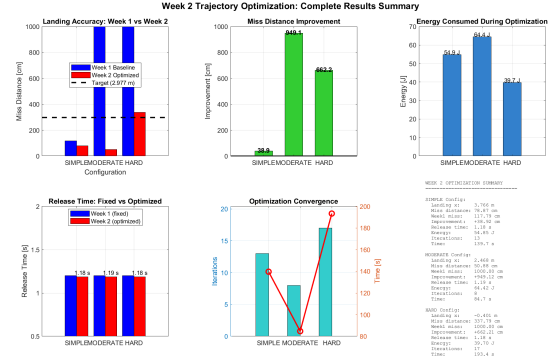


Fig. 7: Week 2 results summary. Top row: landing accuracy, improvement over Week 1, energy consumption. Bottom row: optimized release times, convergence (iterations and time). Moderate achieves the largest improvement (949 cm) over infeasible Week 1 baseline.

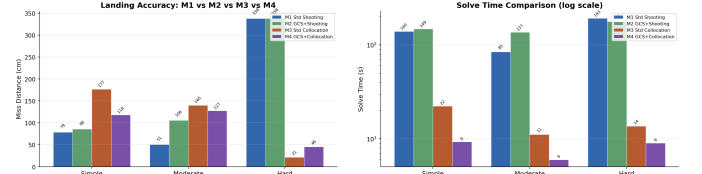


Fig. 8: Left: Miss distance (M1–M4). Right: Solve time (log scale); M4 is fastest at 6–9 s vs. 85–193 s for M1/M2.

H. SCP Collision Analysis

Despite sub-1.3 cm landing accuracy, M5 reports residual arm collisions on moderate (12) and hard (11). Five root causes identified:

1. Discretization gap. $T = 60$, $h \approx 0.033$ s allows inter-step penetration: both $\mathbf{x}_k$ and $\mathbf{x}_{k+1}$ may satisfy constraints while the interpolated midpoint $\frac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1})$ violates them.

TABLE III: Complete results. Bold = best miss per config.

Cfg	Method	Miss (cm)	Time (s)	Iters	Coll
Simple	M1	78.9	140	13	0
	M2	86.0	149	13	0
	M3	177.0	22	679	0
	M4	118.0	9	248	1
	M5	0.7	4	17	0
	M6	6.8	3	13	10
Moderate	M1	50.9	85	8	0
	M2	106.0	137	15	0
	M3	140.0	11	315	0
	M4	127.0	6	174	0
	M5	0.8	5	19	12
	M6	65.9	4	13	0
Hard	M1	337.8	193	17	0
	M2	337.8	178	17	440
	M3	21.0	14	346	0
	M4	45.6	9	202	5
	M5	0.6	5	14	11
	M6	686.7	5	14	0

2. Linearization error. The backward-swing spans $q \in [-25.9, 8.5]$ rad. With trust radius $\rho = 0.5$ rad, the first-order Taylor approximation (17) is poor—circular obstacle geometry is approximated by tangent halfplanes that permit passage in the tangential direction.

3. Sparse geometric sampling. Only 5 points per time step (4 link midpoints + object CoM) are checked, missing link endpoints and intermediate positions.

4. Obstacle-biased initialization. The backward-swing heuristic naturally passes near obstacles placed between the base and target, requiring the SCP to “escape” this collision-prone reference trajectory.

5. QP vs. reality gap. Linearized Forward-Euler dynamics in the QP differ from the RK4-integrated nonlinear trajectory used for post-evaluation.

Mitigation strategies: $T \rightarrow 150$ (eliminates inter-step collisions but $2.5\times$ larger QP); reduce ρ (more iterations); increase δ to 0.15 m (restricts feasible set); collision-aware initialization; `fmincon` post-refinement.

IX. DISCUSSION

A. GCS and Transcription Compatibility

GCS warm-starting benefits collocation (M4) because its $(q_k, \dot{q}_k, \tau_k)$ output maps directly onto the 721-variable collocation structure, placing IPOPT in a favorable basin (42–63% fewer iterations, $1.5\text{--}2.4\times$ speedup, 40J vs. 55J energy on simple). Format conversion for shooting (M2) and SCP (M6) introduces approximation error that degrades rather than helps convergence.

The fundamental asymmetry: *collocation* discretizes both states and controls, matching GCS output naturally. *Shooting* parameterizes only controls; a GCS-derived torque sequence that produces beautiful workspace paths may, when forward-integrated through the nonlinear ODE, produce wildly different states than intended. For *SCP*, the GCS trajectory is not dynamically consistent, so early iterations spend budget on feasibility restoration.

B. Paradigm Trade-offs

Shooting (M1): Fewest variables (41), simplest implementation, but 41 ODE solves per gradient and poor scalability. Fails entirely on hard config (exitflag -2).

Collocation (M3/M4): Exact sparse gradients via automatic differentiation, $10\text{--}100\times$ faster than shooting. Best default choice. M3 achieves the best hard-config result among M1–M4 (21 cm). GCS warm-start (M4) adds $1.5\text{--}2.4\times$ speedup.

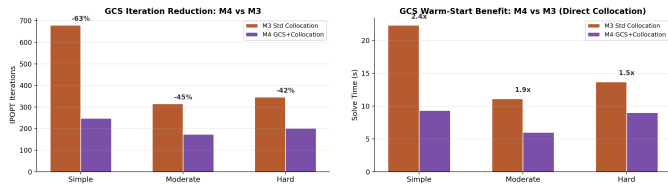


Fig. 9: *Left:* GCS iteration reduction for M4 vs. M3: 42–63% fewer IPOPT iterations. *Right:* Solve time speedup: M4 is $1.5\text{--}2.4\times$ faster.

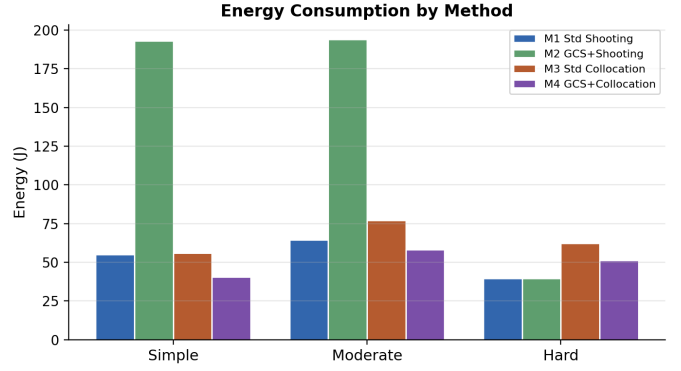


Fig. 10: Energy consumption by method. M2 (GCS+shooting) consumes ≈ 193 J, $3.5\times$ more than M1, due to the GCS initialization dragging the optimizer into a high-energy basin. M4 uses less energy than M3 on all configurations.

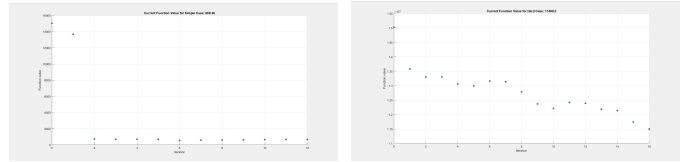


Fig. 11: Shooting convergence: objective value vs. iteration. *Left:* Simple converges from 15000 to 668 in 12 iterations. *Right:* Hard plateaus at ≈ 11500 , indicating a constrained local minimum.

SCP (M5): Highest accuracy (≤ 1.3 cm) through iterative convexification but requires careful tuning. Persistent collision violations on moderate/hard are structural, not parametric: no combination of weights and margins simultaneously achieves sub-2 cm accuracy and zero collisions with $T = 60$.

C. The Accuracy–Collision Trade-off in SCP

High w_{coll} and large δ yield collision-free but inaccurate results (M6 moderate: 0 collisions, 65.9 cm). Lower w_{coll} and moderate δ yield accurate but collision-prone results (M5 moderate: 12 collisions, 0.78 cm). The simple config is the exception: M5 achieves 1.27 cm *and* 0 collisions, indicating both goals can be met when obstacles do not intersect the natural throwing arc.

D. Why Does M5 Beat M3 So Decisively?

M3 achieves 21–177 cm; M5 achieves 0.6–1.3 cm. Three factors: (i) the landing position $x_{\text{land}}(x_T)$ is highly nonlinear, and IPOPT’s barrier method can get trapped in local optima of the 721-variable NLP; (ii) SCP explicitly linearizes and quadratizes the position cost at each iteration, giving quadprog a clean gradient signal; (iii) SCP performs up to 50 iterations of explicit refinement vs. IPOPT’s single-pass NLP.

X. CONCLUSION

We presented a 6-method comparative study of trajectory optimization for a 4-link planar robotic arm throwing task. Key conclusions:

- 1) **SCP (M5)** achieves $\leq 1.3\text{ cm}$ landing error on all configurations ($40\text{--}600\times$ better than M1–M4); residual collisions persist on moderate/hard due to linearization–geometry mismatch.
- 2) **GCS warm-starting** strongly benefits collocation (M4: $1.5\text{--}2.4\times$ faster, $42\text{--}63\%$ fewer IPOPT iterations) but degrades shooting (M2) and is unreliable for SCP (M6).
- 3) **M4** is the recommended default: sparse exact gradients, GCS-accelerated, and robust across all configurations.
- 4) **RK4 rollouts** and **excluding t^* from the QP** (golden-section search instead) are both critical for SCP stability.
- 5) **Collisions in SCP are structural**: five identified root causes require architectural improvements ($T \rightarrow 150$, collision-aware init, post-refinement), not parameter tuning.

Future work: finer SCP discretization ($T = 150$) to eliminate inter-step collisions; collision-aware SCP initialization; 3-D extension via GCS polytope decomposition [6]; automatic convex region generation via IRIS [12].

REFERENCES

- [1] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed. SIAM, 2010.
- [2] M. Kelly, “An introduction to trajectory optimization: How to do your own direct collocation,” *SIAM Review*, vol. 59, no. 4, pp. 849–904, 2017.
- [3] C. R. Hargraves and S. W. Paris, “Direct trajectory optimization using nonlinear programming and collocation,” *J. Guidance*, vol. 10, no. 4, pp. 338–342, 1987.
- [4] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Math. Program.*, vol. 106, pp. 25–57, 2006.
- [5] D. Malyuta *et al.*, “Convex optimization for trajectory generation,” *IEEE Control Syst. Mag.*, vol. 42, no. 5, pp. 40–113, 2022.
- [6] T. Marcucci, M. Petersen, D. von Wrangell, and R. Tedrake, “Motion planning around obstacles with convex optimization,” *Sci. Robot.*, vol. 8, no. 84, 2023.
- [7] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2006.
- [8] A. Zeng *et al.*, “TossingBot: Learning to throw arbitrary objects with residual physics,” *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1307–1319, 2020.
- [9] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [10] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [11] J. A. E. Andersson *et al.*, “CasADi: A software framework for nonlinear optimization and optimal control,” *Math. Prog. Comp.*, vol. 11, pp. 1–36, 2019.
- [12] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Proc. WAFR*, 2015.